Fundamentals of Data Structures

Prof. Pranjal Pandit

Unit 1 Introduction to Algorithms and Data Structures

- From Problem to Program (Problem, Solution, Algorithm, Data Structure and Program).
- Data, Information, Knowledge and Data Structure;
- Abstract Data Types (ADT);
- Classification (Linear/Non-linear, Static/Dynamic, Persistent/Ephemeral).
- Algorithms: problem solving, characteristics, design tools (pseudo-code, flowchart).
- Complexity: space and time complexity, asymptotic notation (Big-O, Theta, Omega),
- Step-count method, analysis of Linear, Quadratic, Cubic and Logarithmic constructs.
- Algorithmic strategies: Divide and Conquer, Greedy.

What is an Algorithmic Strategy?

An **algorithmic strategy** is a general approach or technique used to solve a problem efficiently.

Think of it as a **problem-solving method**. For example:

- If your room is very messy, one strategy is to clean one section at a time.
- Another strategy is to first collect all clothes, then books, then stationery.

Both solve the same problem using different strategies.

Similarly, in computer science, different problems require different algorithmic strategies.

Some common strategies are:

- Brute Force
- Divide and Conquer
- Greedy
- Dynamic Programming
- Backtracking

Divide and Conquer

Definition

Divide and Conquer is a strategy in which a large problem is divided into smaller subproblems, each subproblem is solved independently, and then the solutions are combined to obtain the final answer.

It follows three steps:

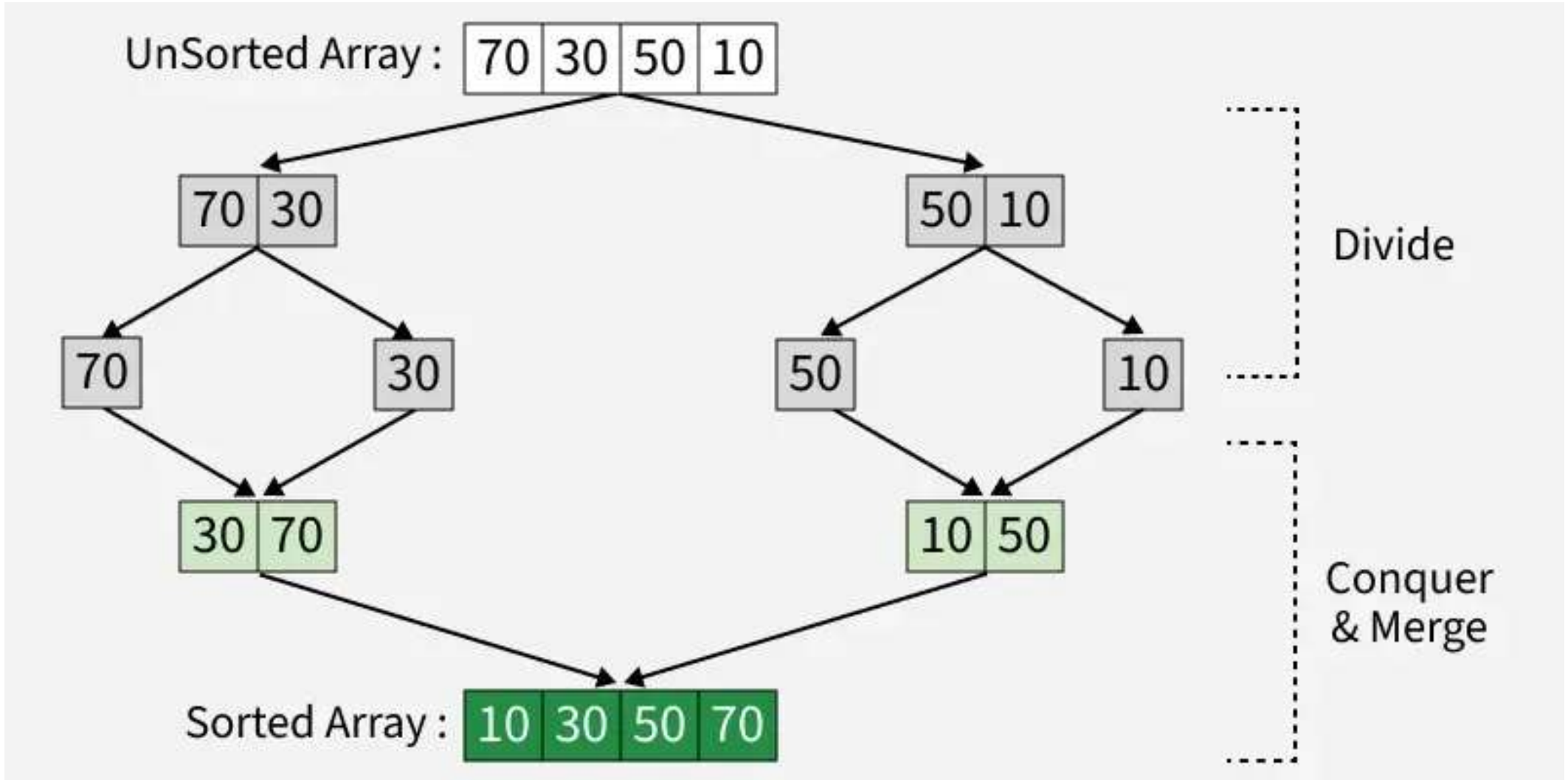
- **Divide:** Break the given problem into smaller non-overlapping problems
- **Conquer:** Solve Smaller Problems
- **Combine:** Use the Solutions of Smaller Problems to find the overall result.

- Examples of Divide and Conquer are Merge Sort, Quick Sort, Binary Search and Closest Pair of Points.

- There is no need of explicit combine step in some algorithms like Binary Search and Quick Sort. Although in Merge Sort, the combine step is the main step.

- The divide step can be trivial in some algorithms (like in Merge Sort and Binary Search, we simply divide in two equal halves). The divide step can be complex in some algorithms like Quick Sort.

- The following image shows working of Divide and Conquer to sort an array using Merge Sort.


UnSorted Array :
70 30 50 10
70 30
50 10
70
30
50
10
30 70
10 50
Sorted Array :
10 30 50 70
Divide
Conquer
& Merge

Greedy Strategy

- Greedy algorithms are a class of algorithms that make locally optimal choices at each step with the hope of finding a global optimum solution.

- A **Greedy algorithm** makes the **best possible choice at the current step** without worrying about future consequences.

- It hopes that these local best choices will lead to the overall best (optimal) solution.

Real-Life Example : Filling a Bag

Bag capacity: 10 kg

Greedy idea: Choose the most valuable item first.

Item	Weight
Gold	1 kg
Silver	3 kg
Iron	8 kg

Computer Example: Dijkstra's Algorithm

Suppose you want the shortest path in a city.

Applications

- Google Maps
- GPS navigation
- Network routing

- Greedy algorithms **do not always give the best solution**. For example, in coin change and 0/1 knapsack problems, we get the best solution using Dynamic Programming.

- Examples of popular algorithms where Greedy gives the best solution are Fractional Knapsack, Dijkstra's algorithm, Kruskal's algorithm, Huffman coding and Prim's Algorithm

Feature	Divide and Conquer	Greedy
Basic Idea	Divide the problem into smaller subproblems, solve them, and combine the results.	Make the best choice at each step and never reconsider it.
Main Steps	Divide → Conquer → Combine	Choose → Move Forward
Subproblems	Yes	Usually No
Revisits Decisions?	Solutions are combined after solving subproblems.	No, once a choice is made, it is not changed.
Usually Uses	Recursion	Iteration (sometimes recursion)
Memory	More	Less
Speed	Efficient for recursive problems	Very efficient for suitable optimization problems
Examples	Binary Search, Merge Sort, Quick Sort	Dijkstra's Algorithm, Huffman Coding, Activity Selection